

우테코 컨벤션

- [자바 코드 컨벤션을 지키면서 프로그래밍했는가?](#)
- [한 메서드에 오직 한 단계의 들여쓰기\(indent\)만 허용했는가?](#)
- [else 예약어를 쓰지 않았는가?](#)
- [모든 원시값과 문자열을 포장했는가?](#)
- [컬렉션에 대해 일급 컬렉션을 적용했는가?](#)
- [3개 이상의 인스턴스 변수를 가진 클래스를 구현하지 않았는가?](#)
- [getter/setter 없이 구현했는가?](#)
- [메소드의 인자 수를 제한했는가?](#)
- [코드 한 줄에 점\(.\)을 하나만 허용했는가?](#)
- [메소드가 한가지 일만 담당하도록 구현했는가?](#)
- [클래스를 작게 유지하기 위해 노력했는가?](#)

자바 코드 컨벤션을 지키면서 프로그래밍했는가?

- 참고
 - <https://google.github.io/styleguide/javaguide.html>
 - <https://myeonguni.tistory.com/1596>
- IntelliJ 또는 Eclipse 통합 개발 도구에서 formatting을 한다.

이건 그냥 "야, 너 자바 처음 하는 티 내지 마!" 이 소리야. 변수 이름은 `myVariable` 처럼, 클래스 이름은 `MyClass` 처럼 쓰라는 아주 기초적인 약속이라고.

한 메서드에 오직 한 단계의 들여쓰기(indent)만 허용했는가?

이게 아주 중요해. 들여쓰기가 깊어진다는 건(if 안에 for, 그 안에 또 if...) 그 메서드가 너무 많은 일을, 너무 복잡한 방식으로 처리하고 있다는 증거야.

- **의미:** 하나의 메서드는 '하나의 추상화 수준'에서 하나의 일만 해야 한다는 원칙(Single Responsibility Principle, SRP)을 강제하는 거야.
- **예시:** 라면을 끓인다 라는 메서드가 있다면, 그 안에는 물을 끓인다, 면과 스프를 넣는다, 계란을 푼다 정도의 단계만 있어야 해. 여기서 물을 끓인다 라는 메서드 안에 '가스 밸브를 연다',

'점화 스위치를 돌린다' 같은 세부 내용이 들어가면 들여쓰기가 깊어지지. 그건 물을 끓인다 라는 별도의 메서드로 분리해야 하는 거야.

- **결과:** 메서드가 짧아지고, 이름만 봐도 무슨 일을 하는지 명확해져.

else 예약어를 쓰지 않았는가?

else 는 코드의 흐름을 두 갈래로 나누는데, 이게 복잡성을 증가시킨다고 보는 거야.

- **의미:** '해피 패스(happy path)', 즉 정상적인 흐름을 주된 로직으로 만들고, 예외적인 상황은 가드 클로저(Guard Clause) 패턴으로 초반에 전부 쳐내라는 뜻이야.
- 예시

```
// 나쁜 예 (else 사용)
public void enterClub(Person person) {
    if (person.isAdult()) {
        // ... 복잡한 입장 로직 ...
    } else {
        System.out.println("미성년자는 출입 금지");
    }
}

// 좋은 예 (else 미사용)
public void enterClub(Person person) {
    if (!person.isAdult()) {
        System.out.println("미성년자는 출입 금지");
        return; // 예외 상황은 먼저 처리하고 끝내버린다.
    }
    // ... 여기부터는 '성인'이라는 게 보장된, 순수한 핵심 로직만 남는다 ...
}
```

모든 원시값과 문자열을 포장했는가?

'원시 타입 집착(Primitive Obsession)'을 피하라는 거야. `int age = 20;` 이나 `String name = "Alice";` 같은 코드는 데이터에 아무런 의미나 제약을 담지 못해.

- **의미:** 단순한 데이터 조각도 그 자체로 의미와 행위를 갖는 '객체'로 만들라는 뜻이야.
- **예시:** `int money` 대신 `Money` 라는 클래스를 만들어. 이 `Money` 클래스는 음수 값을 가질 수 없다는 제약조건을 스스로 가질 수 있고, `add(Money other)` 같은 행위도 정의할 수 있지.
`String` 으로 이메일을 받는 대신 `Email` 클래스를 만들면, 이메일 형식이 올바른지 스스로 검증할 수 있고.

- **결과:** 값과 관련된 비즈니스 로직이 해당 객체 안으로 응집돼. 코드가 훨씬 안전하고 명확해지지. 숫자 100은 그냥 100이지만, '나이' 객체 안의 100은 '100살'이라는 의미와 제약을 갖게 되는 거야. 아름답지 않아?

컬렉션에 대해 일급 컬렉션을 적용했는가?

`List<Car>` 같은 컬렉션을 다른 클래스가 필드로 그냥 갖고 있지 말라는 의미야.

- **의미:** 컬렉션을 그 자체로 포장해서, 컬렉션과 관련된 로직을 그 포장한 클래스가 전부 책임지도록 하는 거야.
- **예시:** `Game` 클래스가 `List<Player>` 를 갖는 대신, `Players` 라는 클래스를 만들어. 그리고 그 `Players` 클래스가 내부에 `List<Player>` 를 갖는 거지. '우승자를 찾는다'거나 '모든 플레이어의 점수를 계산한다' 같은 로직은 이제 `Game` 이 아니라 `Players` 클래스의 책임이 되는 거야.
- **결과:** 컬렉션과 관련된 상태와 행위가 하나의 클래스로 묶여서 관리돼. 이것도 응집도를 높이는 기술이야.

3개 이상의 인스턴스 변수를 가진 클래스를 구현하지 않았는가?

- 쉽지 않은 연습일 수 있다. 가능하면 인스턴스 변수의 수를 줄이기 위해 노력한다.
- 클래스가 너무 많은 책임을 지려고 할 때 나타나는 첫 번째 신호가 바로 인스턴스 변수의 개수야.
 - **의미:** 클래스는 가능한 작고, 하나의 책임만 져야 한다는 SRP를 극단적으로 강제하는 규칙이야. 변수가 3~4개 이상이 된다는 건, 그 클래스가 여러 개념을 동시에 다루고 있을 가능성이 높다는 신호지.
 - **조치:** 변수들을 논리적으로 묶어서 별도의 작은 클래스로 분리해야 해. 예를 들어 `x좌표`, `y좌표` 변수가 있다면 `Position` 이라는 객체로 묶는 거지.
 - **결과:** 작고 응집도 높은 클래스들이 만들어져. 레고 블록처럼 말이야. 크고 복잡한 덩어리 하나보다, 작고 단순한 블록 여러 개가 훨씬 조합하기 좋잖아?

getter/setter 없이 구현했는가?

- 핵심 로직을 구현하는 도메인 객체에 `getter/setter`를 쓰지 않고 구현했는가?
- 단, DTO는 허용한다.

| 묻지 말고 시켜라. (Tell, Don't ask)

이게 객체지향의 핵심이야.

- **의미:** 객체의 내부 상태(변수)를 `get` 으로 꺼내와서 다른 곳에서 판단하고 `set` 으로 바꾸지 마. 그건 객체를 그냥 데이터 저장 가방으로 쓰는 거야. 대신, 객체에게 **"이 일을 해줘"** 라고 메시지를 보내라는 거야. 상태를 바꾸는 책임은 전적으로 객체 자신이 져야 해.

- ```
// 나쁜 예 (묻고 판단하기)
if (player.getHp() <= 0) {
 player.setStatus(Status.DEAD);
}

// 좋은 예 (시켜서 스스로 판단하게 하기)
player.takeDamage(100); // player 객체 내부에서 스스로 hp를 깎고, 0 이하면 상태를 DEAD로 바꾼다.
```

---

## 메소드의 인자 수를 제한했는가?

---

- 4개 이상의 인자는 허용하지 않는다.
- 3개도 가능하면 줄이기 위해 노력해 본다.

인자가 4개 이상이라는 건 그 메서드가 너무 많은 정보를 필요로 한다는 뜻. 즉, 너무 복잡한 일을 하려 하거나, 관련 없는 정보들을 한 번에 처리하려 한다는 신호야.

- **의미:** 메서드의 복잡도를 낮추고, 파라미터들을 객체로 묶어서 개념을 명확히 하라는 거야.
- **조치:** `drawCircle(int x, int y, int radius, String color)` 대신, `drawCircle(CircleData data)` 처럼 관련된 인자들을 하나의 객체로 묶어서 전달하는 걸 고려해봐.
- **결과:** 메서드 시그니처가 깔끔해지고, 데이터의 연관 관계가 명확해져.

---

## 코드 한 줄에 점(.)을 하나만 허용했는가?

---

- 디미터(Demeter)의 법칙("친구하고만 대화하라")을 지켰는가?
- 예를 들어 `location.current.representation.substring(0, 1)` 와 같이 여러 개의 점( . )이 등장하면 리팩토링할 부분을 찾아본다.

이건 **디미터 법칙(Law of Demeter)**, "친구의 친구에게 말 걸지 마라"는 규칙이야.

- **의미:** `user.getWallet().getMoney().add(100)` 같은 코드는 `user` 가 `wallet` 의 내부를, `wallet` 이 `Money` 의 내부를 너무 잘 알고 있다는 뜻이야. 객체들 간의 결합도가 너무 높아지는 거지.

- **조치:** `user`에게 "네 지갑에 100원 넣어줘" 라고 시켜야 해. `user.deposit(100);` 처럼. 돈을 넣는 구체적인 방법은 `user`가 알아서 `wallet`에게 시키고, `wallet`이 `Money`에게 시키는 식으로 위임해야지.
  - **결과:** 객체 간의 결합도가 낮아져서 한 객체의 내부 구조가 바뀌어도 다른 객체에 미치는 영향이 적어져. 유지보수가 훨씬 쉬워지지.
- 

## 메소드가 한가지 일만 담당하도록 구현했는가?

---

함수 하나한테 '밥도 하고, 청소도 하고, 빨래도 하고, 숙제도 해!' 이렇게 시키면 제대로 하겠냐? 너처럼 다 망쳐놓겠지. 함수는 무조건 딱 한 가지 일만 시켜야 해. 이름도 `doSomethingAndSomethingElse` 이딴 식으로 짓지 말고. '밥하기'면 '밥하기', '청소하기'면 '청소하기'. 단순하게.

---

## 클래스를 작게 유지하기 위해 노력했는가?

---

이건 그냥 '클래스를 작게 만들어라'는 소리를 한번 더 하는 거야. 중요한 거니까. 클래스가 커지고 코드가 길어지면, 너 같은 애들은 10줄만 넘어가도 위에서 뭘 썼는지 까먹잖아. 그러니까 잘게, 아주 잘게 쪼개라는 거야. 레고 블록처럼. 그래야 네가 감당할 수 있으니까.